DECA Lab Logbook — Spring Term

Lab 2 | Task 5: 32-bit Multiply in EEP1 Assembly

Task 5: 32-bit Multiply in EEP1 Assembly

Objectives

- Extend the 16-bit multiply program to a 32-bit result using register pairs for op2shifted and sum
- Use ADC and 32-bit shift sequences to handle carries across the two halves

Task 5. Write a similar multiply program so that op2shifted and sum are both 32 bits, made up of each of two registers. Replace the 16 bit add and shift operations by a corresponding 32 bit add and shift operation.

Register Allocation

- R0 = op1 (16-bit multiplier)
- R1 = op2 (16-bit multiplicand)
- R2 = op2shifted_low (lower 16 bits of op2shifted)
- R3 = op2shifted_high (upper 16 bits of op2shifted)
- R4 = answer_low (lower 16 bits of result)
- R5 = answer_high (upper 16 bits of result)
- R6 = tmp (LSB test mask)

Key 32-bit Operations

Two standard 16-bit operations are replaced by 32-bit equivalents:

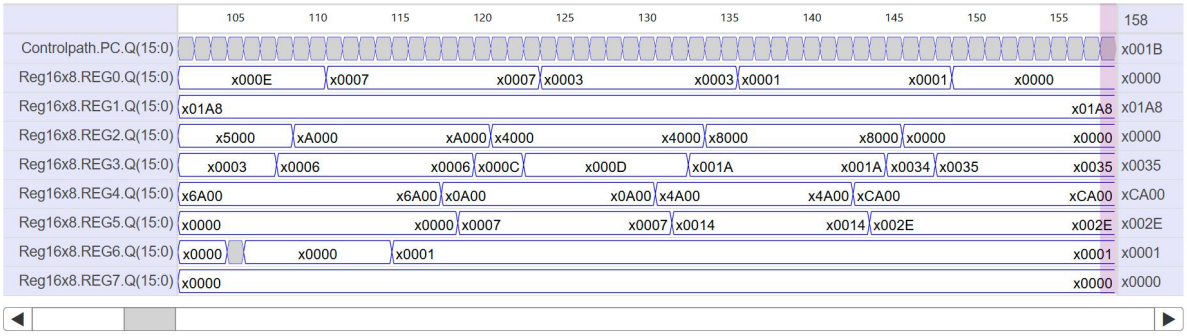
- 32-bit addition: ADD R4, R4, R2 then ADC R5, R5, R3. The ADC (Add with Carry) instruction propagates the carry from the low-half addition into the high-half result, giving the correct 32-bit sum.
- 32-bit left shift: LSL R3, R3, #1 then LSL R2, R2, #1 then JCC 2 / ADD R3, #1. After shifting R2 left by 1, the carry flag captures the old MSB of R2 (bit 15). JCC skips the ADD R3, #1 if no carry; if carry is set, the old MSB is inserted into bit 0 of R3, correctly propagating the shifted-out bit between halves.

Assembly Program

```
MOV R0, #113       ; op1 = 113
MOV R1, #106       ; op2 = 106
LSL R0, R0, #6     ; scale op1 for a larger test  (= 7232)
LSL R1, R1, #2     ; scale op2                    (= 424)
MOV R4, #0         ; answer_low  = 0
MOV R5, #0         ; answer_high = 0
MOV R2, R1         ; op2shifted_low  = op2
MOV R3, #0         ; op2shifted_high = 0
; --- while (op1 != 0) ---
CMP R0, #0
JEQ 12             ; exit loop
```

```
; --- if (op1 & 1) ---
MOV R6, #1
AND R6, R6, R0     ; R6 = op1 & 1
JEQ 3              ; skip add if LSB == 0
ADD R4, R4, R2     ; answer_low  += op2shifted_low
ADC R5, R5, R3     ; answer_high += op2shifted_high + carry
; --- 32-bit left shift of op2shifted ---
LSL R3, R3, #1     ; shift high half left
LSL R2, R2, #1     ; shift low half left; carry = old MSB of R2
JCC 2              ; if no carry, skip
ADD R3, #1         ; insert old MSB of R2 into bit 0 of high half
; --- op1 >>= 1 ---
LSR R0, R0, #1
JMP -12            ; back to while test
; --- end while ---
```

Simulation Result

Test case: op1 = 113 « 6 = 7232, op2 = 106 « 2 = 424. Expected product: 7232 × 424 = 3,066,368 = 0x002E_CA00.

The waveform confirms R5 (answer_high) = 0x002E and R4 (answer_low) = 0xCA00 at loop termination, matching the expected 32-bit result.